

Лабораторная работа №13

Алгоритмы Прима и Краскала. Минимальное остовное дерево

Цель работы:

1. Изучить понятие минимального остовного дерева (Minimum Spanning Tree, MST) и его применение
2. Реализовать алгоритм Прима для построения MST в связном взвешенном графе
3. Реализовать алгоритм Краскала для построения MST с использованием системы непересекающихся множеств (DSU)
4. Сравнить эффективность алгоритмов Прима и Краскала на различных типах графов
5. Научиться применять MST для решения практических задач (сети связи, дорожные сети)

Теоретическое обоснование

1. Понятие минимального остовного дерева

Остовное дерево (Spanning Tree) - это подграф связного графа, который:

Содержит все вершины исходного графа

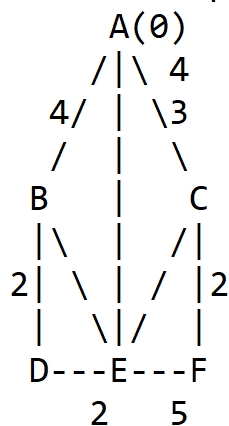
Является деревом (не содержит циклов)

Имеет ровно $V-1$ рёбер

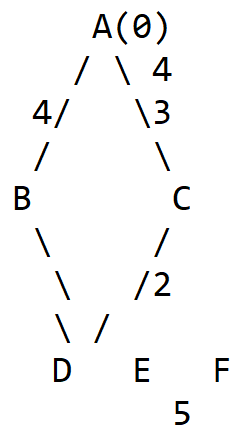
Минимальное остовное дерево (Minimum Spanning Tree, MST) - остовное дерево с минимальной суммой весов рёбер.

Визуализация:

Исходный граф:



Минимальное остовное дерево:



Вес исходного графа: $4+3+4+2+2+5+\dots = 20+$

Вес MST: $4+3+2+2+5 = 16$

2. Свойства минимального остовного дерева

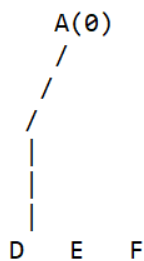
Свойство	Описание
Единственность	Если все веса рёбер различны, MST единственно
Разрез	Минимальное ребро, пересекающее разрез, принадлежит MST
Цикл	Максимальное ребро в любом цикле не принадлежит MST

3. Алгоритм Прима (Prim's Algorithm)

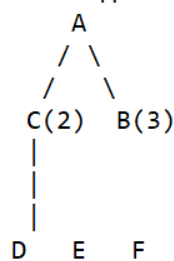
Принцип работы: Жадный алгоритм, который начинает с одной вершины и на каждом шаге добавляет минимальное ребро, соединяющее текущее дерево с ещё не включённой вершиной.

Визуализация алгоритма Прима:

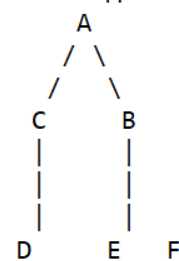
Шаг 1: Начинаем с A



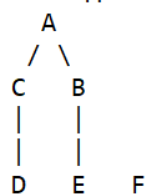
Шаг 2: Добавляем C (2)



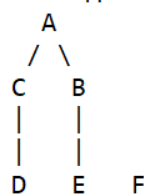
Шаг 3: Добавляем B (3)



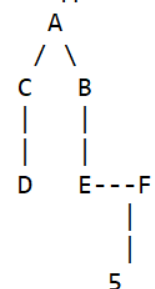
Шаг 4: Добавляем D (4)



Шаг 5: Добавляем E (2)



Шаг 6: Добавляем F (5)



Алгоритм:

```

int primMST(int start) {
    vector<int> minEdge(V, INF); // минимальное ребро для каждой
    // вершины
    vector<bool> inMST(V, false); // входит ли вершина в MST
    vector<int> parent(V, -1); // родитель в MST
    int totalWeight = 0;

    minEdge[start] = 0;
  
```

```

for (int i = 0; i < V; i++) {
    // Находим вершину с минимальным minEdge среди не включённых
    int u = -1;
    for (int j = 0; j < V; j++) {
        if (!inMST[j] && (u == -1 || minEdge[j] < minEdge[u])) {
            u = j;
        }
    }

    inMST[u] = true;
    totalWeight += minEdge[u];

    // Обновляем соседей
    for (int v = 0; v < V; v++) {
        if (adj[u][v] != 0 && !inMST[v] && adj[u][v] < minEdge[v]) {
            minEdge[v] = adj[u][v];
            parent[v] = u;
        }
    }
}

return totalWeight;
}

```

Оптимизированная версия с priority_queue:

```

int primMST_pq(int start) {
    vector<int> minEdge(V, INF);
    vector<bool> inMST(V, false);
    vector<int> parent(V, -1);
    int totalWeight = 0;

    using pii = pair<int, int>; // {вес, вершина}
    priority_queue<pii, vector<pii>, greater<pii>> pq;

    minEdge[start] = 0;
    pq.push({0, start});

    while (!pq.empty()) {
        int w = pq.top().first;
        int u = pq.top().second;
        pq.pop();
    }
}

```

```

    if (inMST[u]) continue;

    inMST[u] = true;
    totalWeight += w;

    for (int v = 0; v < V; v++) {
        if (adj[u][v] != 0 && !inMST[v] && adj[u][v] < minEdge[v]) {
            minEdge[v] = adj[u][v];
            parent[v] = u;
            pq.push({minEdge[v], v});
        }
    }
}

return totalWeight;
}

```

4. Алгоритм Краскала (Kruskal's Algorithm)

Принцип работы: Сортирует все рёбра по весу и добавляет их в MST, если они не создают цикл (проверка через DSU).

Визуализация алгоритма Краскала:

Список рёбер, отсортированный по весу:

1. (A,B): 4 6. (E,F): 5
2. (C,D): 2 7. (D,E): 5
3. (B,C): 3 8. (A,F): 6
4. (B,D): 2 9. (C,E): 7
5. (A,C): 3

Шаг 1: (C,D):2
C-D

Шаг 2: (B,D):2
C-D
|
B

Шаг 3: (B,C):3
C-D
|
B-C
|
D

Шаг 4: (A,C):3
A-C-D
|
B

Шаг 5: (E,F):5
A-C-D
|
B
|
E-F

Алгоритм:

```
struct Edge {
    int u, v, weight;
    Edge(int u, int v, int w) : u(u), v(v), weight(w) {}
    bool operator<(const Edge& other) const {
        return weight < other.weight;
    }
};

int kruskalMST(vector<Edge>& edges) {
    // Сортировка рёбер по весу
    sort(edges.begin(), edges.end());

    DSU dsu(V);
    int totalWeight = 0;
    vector<Edge> mstEdges;

    for (const Edge& e : edges) {
        if (dsu.find(e.u) != dsu.find(e.v)) {
            dsu.unite(e.u, e.v);
            totalWeight += e.weight;
            mstEdges.push_back(e);
            if (mstEdges.size() == V - 1) break;
        }
    }

    return totalWeight;
}
```

5. Система непересекающихся множеств (DSU)

DSU (Disjoint Set Union) - структура данных для объединения множеств и проверки принадлежности элементов одному множеству.

```
class DSU {
private:
    vector<int> parent;
    vector<int> rank;

public:
    DSU(int n) {
```

```

    parent.resize(n);
    rank.resize(n, 0);
    for (int i = 0; i < n; i++) {
        parent[i] = i;
    }
}

int find(int x) {
    if (parent[x] != x) {
        parent[x] = find(parent[x]); // сжатие пути
    }
    return parent[x];
}

void unite(int x, int y) {
    int rootX = find(x);
    int rootY = find(y);

    if (rootX == rootY) return;

    // объединение по рангу
    if (rank[rootX] < rank[rootY]) {
        parent[rootX] = rootY;
    } else if (rank[rootX] > rank[rootY]) {
        parent[rootY] = rootX;
    } else {
        parent[rootY] = rootX;
        rank[rootX]++;
    }
}

bool connected(int x, int y) {
    return find(x) == find(y);
}
};

```

6. Сравнение алгоритмов

Характеристика	Алгоритм Прима	Алгоритм Краскала
Тип графа	Неориентированный	Неориентированный

Характеристика	Алгоритм Прима	Алгоритм Краскала
Сложность (простая)	$O(V^2)$	$O(E \log E)$
Сложность (оптимизированная)	$O((V+E) \log V)$	$O(E \log E)$
Подход	Наращивание дерева	Объединение компонент
Лучше для	Плотных графов	Разреженных графов
Память	$O(V)$ + матрица	$O(E)$
Структура данных	Очередь с приоритетами	DSU + сортировка

Примеры выполнения практической части

Пример 1. Алгоритм Прима (базовый уровень)

Задача: Реализовать алгоритм Прима для нахождения минимального остовного дерева.

```
#include <iostream>
#include <vector>
#include <climits>

using namespace std;

class PrimMST {
private:
    int V;
    vector<vector<int>>> graph;

public:
    PrimMST(int vertices) : V(vertices) {
        graph.resize(V, vector<int>(V, 0));
    }

    void addEdge(int u, int v, int weight) {
        graph[u][v] = weight;
        graph[v][u] = weight;
    }
};
```

```

    }

    int primMST() {
        vector<int> minEdge(V, INT_MAX);
        vector<bool> inMST(V, false);
        vector<int> parent(V, -1);
        int totalWeight = 0;

        minEdge[0] = 0;

        for (int i = 0; i < V; i++) {
            // Находим вершину с минимальным minEdge
            int u = -1;
            for (int j = 0; j < V; j++) {
                if (!inMST[j] && (u == -1 || minEdge[j] < minEdge[u])) {
                    u = j;
                }
            }

            if (minEdge[u] == INT_MAX) {
                cout << "Граф не связный!" << endl;
                return -1;
            }

            inMST[u] = true;
            totalWeight += minEdge[u];

            // Обновляем соседей
            for (int v = 0; v < V; v++) {
                if (graph[u][v] != 0 && !inMST[v] && graph[u][v] < minEdge[v])
                {
                    minEdge[v] = graph[u][v];
                    parent[v] = u;
                }
            }
        }

        // Вывод рёбер MST
        cout << "Рёбра минимального остовного дерева:" << endl;
        for (int i = 1; i < V; i++) {
            cout << " " << parent[i] << " - " << i << " (вес: " << minEdge[i] <<
            ")" << endl;
        }
    }
}

```

```

    }

    return totalWeight;
}

};

int main() {
    PrimMST g(5);

    g.addEdge(0, 1, 2);
    g.addEdge(0, 3, 6);
    g.addEdge(1, 2, 3);
    g.addEdge(1, 3, 8);
    g.addEdge(1, 4, 5);
    g.addEdge(2, 4, 7);
    g.addEdge(3, 4, 9);

    int weight = g.primMST();
    cout << "Общий вес MST: " << weight << endl;

    return 0;
}

```

Ожидаемый вывод:

Рёбра минимального остовного дерева:

0 - 1 (вес: 2)

1 - 2 (вес: 3)

1 - 4 (вес: 5)

0 - 3 (вес: 6)

Общий вес MST: 16

Пример 2. Алгоритм Краскала (средний уровень)

Задача: Реализовать алгоритм Краскала с использованием DSU.

```

#include <iostream>
#include <vector>
#include <algorithm>

```

```
using namespace std;

```

```

class DSU {
private:
    vector<int> parent;

```

```
vector<int> rank;
```

```
public:
```

```
DSU(int n) {  
    parent.resize(n);  
    rank.resize(n, 0);  
    for (int i = 0; i < n; i++) {  
        parent[i] = i;  
    }  
}
```

```
int find(int x) {  
    if (parent[x] != x) {  
        parent[x] = find(parent[x]);  
    }  
    return parent[x];  
}
```

```
void unite(int x, int y) {  
    int rootX = find(x);  
    int rootY = find(y);  
  
    if (rootX == rootY) return;  
  
    if (rank[rootX] < rank[rootY]) {  
        parent[rootX] = rootY;  
    } else if (rank[rootX] > rank[rootY]) {  
        parent[rootY] = rootX;  
    } else {  
        parent[rootY] = rootX;  
        rank[rootX]++;  
    }  
}
```

```
bool connected(int x, int y) {  
    return find(x) == find(y);  
}  
};
```

```
struct Edge {  
    int u, v, weight;  
    Edge(int u, int v, int w) : u(u), v(v), weight(w) {}  
};
```

```

};

class KruskalMST {
private:
    int V;
    vector<Edge> edges;

public:
    KruskalMST(int vertices) : V(vertices) {}

    void addEdge(int u, int v, int weight) {
        edges.push_back(Edge(u, v, weight));
    }

    int kruskalMST() {
        // Сортировка рёбер по весу
        sort(edges.begin(), edges.end(),
            [](const Edge& a, const Edge& b) {
                return a.weight < b.weight;
            });

        DSU dsu(V);
        int totalWeight = 0;
        int edgesUsed = 0;

        cout << "Рёбра минимального остовного дерева:" << endl;

        for (const Edge& e : edges) {
            if (!dsu.connected(e.u, e.v)) {
                dsu.unite(e.u, e.v);
                totalWeight += e.weight;
                edgesUsed++;
                cout << " " << e.u << " - " << e.v << " (Вес: " << e.weight << ")"
                << endl;

                if (edgesUsed == V - 1) break;
            }
        }

        if (edgesUsed != V - 1) {
            cout << "Граф не связный!" << endl;
            return -1;
        }
    }
};

```

```

    }

    return totalWeight;
}

};

int main() {
    KruskalMST g(5);

    g.addEdge(0, 1, 2);
    g.addEdge(0, 3, 6);
    g.addEdge(1, 2, 3);
    g.addEdge(1, 3, 8);
    g.addEdge(1, 4, 5);
    g.addEdge(2, 4, 7);
    g.addEdge(3, 4, 9);

    int weight = g.kruskalMST();
    cout << "Общий вес MST: " << weight << endl;

    return 0;
}

```

Ожидаемый вывод:

Рёбра минимального остовного дерева:

0 - 1 (вес: 2)

1 - 2 (вес: 3)

1 - 4 (вес: 5)

0 - 3 (вес: 6)

Общий вес MST: 16

Пример 3. Сравнение алгоритмов (продвинутый уровень)

Задача: Сравнить производительность алгоритмов Прима и Краскала на графах разного размера.

```

#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>

```

```

using namespace std;
using namespace chrono;

```

```

class GraphComparison {
private:
    int V;
    vector<vector<int>> adjMatrix;
    vector<tuple<int, int, int>> edgeList;

public:
    GraphComparison(int vertices) : V(vertices) {
        adjMatrix.resize(V, vector<int>(V, 0));
    }

    void generateRandomGraph(int edgeDensity) {
        mt19937 rng(42);
        uniform_int_distribution<int> weightDist(1, 100);
        uniform_int_distribution<int> edgeDist(0, 100);

        for (int i = 0; i < V; i++) {
            for (int j = i + 1; j < V; j++) {
                if (edgeDist(rng) < edgeDensity) {
                    int w = weightDist(rng);
                    adjMatrix[i][j] = w;
                    adjMatrix[j][i] = w;
                    edgeList.push_back({i, j, w});
                }
            }
        }
    }

    int primMST() {
        vector<int> minEdge(V, INT_MAX);
        vector<bool> inMST(V, false);
        minEdge[0] = 0;
        int totalWeight = 0;

        for (int i = 0; i < V; i++) {
            int u = -1;
            for (int j = 0; j < V; j++) {
                if (!inMST[j] && (u == -1 || minEdge[j] < minEdge[u])) {
                    u = j;
                }
            }
            if (minEdge[u] == INT_MAX) return -1;

```

```

        inMST[u] = true;
        totalWeight += minEdge[u];
        for (int v = 0; v < V; v++) {
            if (adjMatrix[u][v] != 0 && !inMST[v] && adjMatrix[u][v] <
minEdge[v]) {
                minEdge[v] = adjMatrix[u][v];
            }
        }
    }
    return totalWeight;
}

```

```

int kruskalMST() {
    vector<tuple<int, int, int>> edges = edgeList;
    sort(edges.begin(), edges.end(), [](const auto& a, const auto& b) {
        return get<2>(a) < get<2>(b);
    });
}

```

```

vector<int> parent(V);
for (int i = 0; i < V; i++) parent[i] = i;

```

```

function<int(int)> find = [&](int x) {
    if (parent[x] != x) parent[x] = find(parent[x]);
    return parent[x];
};

```

```

int totalWeight = 0;
int edgesUsed = 0;

```

```

for (const auto& e : edges) {
    int u = get<0>(e), v = get<1>(e), w = get<2>(e);
    if (find(u) != find(v)) {
        parent[find(u)] = find(v);
        totalWeight += w;
        edgesUsed++;
        if (edgesUsed == V - 1) break;
    }
}

```

```

return edgesUsed == V - 1 ? totalWeight : -1;
}

```

```

void compare(int edgeDensity) {
    generateRandomGraph(edgeDensity);

    auto start = high_resolution_clock::now();
    int primWeight = primMST();
    auto end = high_resolution_clock::now();
    auto primTime = duration_cast<microseconds>(end - start).count();

    start = high_resolution_clock::now();
    int kruskalWeight = kruskalMST();
    end = high_resolution_clock::now();
    auto kruskalTime = duration_cast<microseconds>(end - start).count();

    cout << "V = " << V << ", плотность = " << edgeDensity << "%" <<
endl
    cout << " Прим: время = " << primTime << " мкс, вес = " <<
primWeight << endl
    cout << " Краскал: время = " << kruskalTime << " мкс, вес = " <<
kruskalWeight << endl;
    cout << " Отношение (Краскал/Прим): " << (double)kruskalTime /
primTime << endl;
    cout << endl;
}
};

int main() {
    cout << "=== Сравнение алгоритмов Прима и Краскала ===" << endl
<< endl

    // Разреженный граф (20% рёбер)
    GraphComparison sparse(100);
    sparse compare(20);

    // Плотный граф (60% рёбер)
    GraphComparison dense(100);
    dense compare(60);

    // Очень плотный граф (90% рёбер)
    GraphComparison veryDense(100);
    veryDense compare(90);

    return 0;
}

```

Индивидуальные задания

Часть 1. Алгоритм Прима

Каждый вариант содержит задание реализовать алгоритм Прима с указанными особенностями:

№ вар.	Способ реализации	Особенности
1	Без оптимизации ($O(V^2)$)	Неориентированный граф
2	С priority_queue ($O((V+E) \log V)$)	Неориентированный граф
3	Без оптимизации ($O(V^2)$)	С восстановлением дерева
4	С priority_queue ($O((V+E) \log V)$)	С восстановлением дерева
5	Без оптимизации ($O(V^2)$)	Разреженный граф
6	С priority_queue ($O((V+E) \log V)$)	Плотный граф
7	Без оптимизации ($O(V^2)$)	Поиск MST с указанной начальной вершиной
8	С priority_queue ($O((V+E) \log V)$)	Поиск MST с указанной начальной вершиной
9	Без оптимизации ($O(V^2)$)	Вывод рёбер в порядке добавления
10	С priority_queue ($O((V+E) \log V)$)	Вывод рёбер в порядке добавления
11	Без оптимизации ($O(V^2)$)	Граф с целыми весами
12	С priority_queue ($O((V+E) \log V)$)	Граф с вещественными весами
13	Без оптимизации ($O(V^2)$)	Граф с большим количеством вершин ($V=500$)

№ вар.	Способ реализации	Особенности
14	С priority_queue ($O((V+E) \log V)$)	Граф с большим количеством вершин ($V=500$)
15	Оба варианта	Сравнение производительности

Часть 2. Алгоритм Краскала

Каждый вариант содержит задание реализовать алгоритм Краскала с указанными особенностями:

№ вар.	Особенности	Дополнительное задание
1	Простая сортировка	Вывод рёбер в порядке добавления
2	С использованием DSU	Вывод рёбер в порядке добавления
3	Простая сортировка	Проверка связности графа
4	С использованием DSU	Проверка связности графа
5	Простая сортировка	Разреженный граф
6	С использованием DSU	Плотный граф
7	Простая сортировка	С вещественными весами
8	С использованием DSU	С вещественными весами
9	Простая сортировка	Граф с большим количеством вершин ($V=1000$)
10	С использованием DSU	Граф с большим количеством вершин ($V=1000$)
11	Простая сортировка	Поиск второго минимального MST

№ вар.	Особенности	Дополнительное задание
12	С использованием DSU	Поиск второго минимального MST
13	Простая сортировка	Обработка нескольких компонент связности
14	С использованием DSU	Обработка нескольких компонент связности
15	Оба варианта	Сравнение эффективности DSU с рангами и без

Часть 3. Применение MST

Задание для всех вариантов:

Реализовать программу для решения задачи о прокладке сетей. Дано N городов и M возможных дорог между ними с указанием стоимости строительства.

Найти минимальную стоимость соединения всех городов (MST)

Вывести список дорог, которые нужно построить

Если граф не связный, вывести компоненты связности

Пример входных данных:

Города: A, B, C, D, E, F

Дороги:

A-B: 4

A-C: 3

B-C: 2

B-D: 2

C-E: 3

D-E: 5

E-F: 5

Ожидаемый результат:

Минимальная стоимость: 16

Дороги для строительства:

B-C (2)

B-D (2)

A-C (3)

C-E (3)

Контрольные вопросы

1. Что такое минимальное остовное дерево (MST)?
2. Какие свойства имеет минимальное остовное дерево?
3. В чём заключается идея алгоритма Прима?
4. Какова временная сложность алгоритма Прима на матрице смежности?
5. Какова временная сложность алгоритма Прима с приоритетной очередью?
6. В чём заключается идея алгоритма Краскала?
7. Что такое DSU и как она используется в алгоритме Краскала?
8. Какова временная сложность алгоритма Краскала?
9. В каких случаях алгоритм Прима эффективнее алгоритма Краскала?
10. В каких случаях алгоритм Краскала эффективнее алгоритма Прима?
11. Как проверить, что построенное дерево является минимальным?
12. Что произойдёт, если граф не связный?
13. Как модифицировать алгоритмы для поиска максимального остовного дерева?
14. Где применяются MST в реальной жизни?
15. Как найти второй по минимальности вес MST?

Требования к отчёту

Титульный лист

Цель работы

Теоретическая часть (MST, Прим, Краскал, DSU)

Постановка задачи (согласно варианту)

Листинг программы с комментариями

Результаты работы (скриншоты вывода)

Анализ сложности алгоритмов

Сравнительная таблица производительности

Ответы на контрольные вопросы

Выводы

Ссылка на Git-репозиторий

Критерии оценивания

Максимальный балл: 100

№	Критерий	Баллы
1	Алгоритм Прима	30
1.1	Реализация алгоритма	15
1.2	Корректность работы	10
1.3	Восстановление дерева (по варианту)	5
2	Алгоритм Краскала	35
2.1	Реализация алгоритма	15
2.2	Реализация DSU	10
2.3	Корректность работы	10
3	Применение и анализ	20
3.1	Решение прикладной задачи	10
3.2	Сравнение производительности	5
3.3	Выводы о применимости	5
4	Оформление и отчёт	15
4.1	Код отформатирован, есть комментарии	5
4.2	Есть ответы на контрольные вопросы	5
4.3	Ссылка на Git-репозиторий	5
Итого		100

Штрафные баллы

Нарушение	Штраф
Неправильная работа DSU (циклы)	-15
Неверная сложность алгоритма	-10
Отсутствие обработки несвязного графа	-10
Нет сравнения производительности	-10
Нет ссылки на Git-репозиторий	-10
Программа не компилируется	работа не принимается

Итоговая шкала

Оценка	Баллы
Отлично	90–100
Хорошо	75–89
Удовлетворительно	60–74
Неудовлетворительно	менее 60